ChatGpt Client-ChatGpt Style Interface with Flutter & Riverpod

Alogrithm / Methodology

System Overview:

The System is a mobile chat application where

- User sends a message
- App sends request to AI API
- AI generates response
- Response is displayed in Chat UI

Step By Step Alogrithm:

Step1: Initialize App

- Start Flutter app
- Initialize Riverpod providers
- Setup theme

Step2: Create State Management (Riverpod)

- messageProvider -Stores chat messages list
- apiServiceProvider - handles API calls
- loadingProvider -tracks response loading

Step3: User Input Haandling

1. User types message in text field
2. On “Send” button click
    - Validate input
    - Add message to chat list
    - Clear input field

Step4: API Request Processing

i. Set loading=true
ii. Send HTTP request to PI API
iii. Include:
    - User message

- Previous message

Step5: Response Handling

- Receive API response
- Extract AI reply
- Add reply to message list
- Set loading = false

Step6: UI Update

- Riverpod automatically rebuilds UI
- Chat screen updates with:
- User message(right side)
- AI response (left side)
- Show typing indicator while loading

Step7: Error Handling

If API fails:

- Show error message
- Retry option
- Stop loading

Logic or Method Used:

Uses client –server architecture

Asynchronous API calls for real-time response

State management handled using Riverpod

UI updates dynamically based on state changes

Model or Technique Applied:

Natural Language Processing (NLP)

Transformer-based language model (via API)

REST API integration for communication

Reactive programming for UI updates

Process Flow of System:

User Input

↓

Flutter UI (TextField)

↓

Riverpod State Update

↓

API Call (HTTP Request)

↓

Backend / AI Model

↓

Response Generation

↓

Return JSON Response

↓

Riverpod Updates State

↓

UI Rebuild (Chat Bubble Display)

Process flow Explanation:

The process flow of a ChatGPT-style app using Flutter and Riverpod is simple and works in a loop. When the user opens the app, the chat screen is displayed with a text box and send button. The user types a message and sends it, and the message is immediately shown on the screen using Riverpod state management. At the same time, the app sends this message to an AI API in the background. While waiting, a loading or typing indicator is shown. Once the AI generates a response, it is received by the app and added to the chat, and the UI

updates automatically. If there is any error, a message is shown to the user. This process repeats every time the user sends a new message, creating a smooth chat experience like ChatGPT.

Dataset of the project:

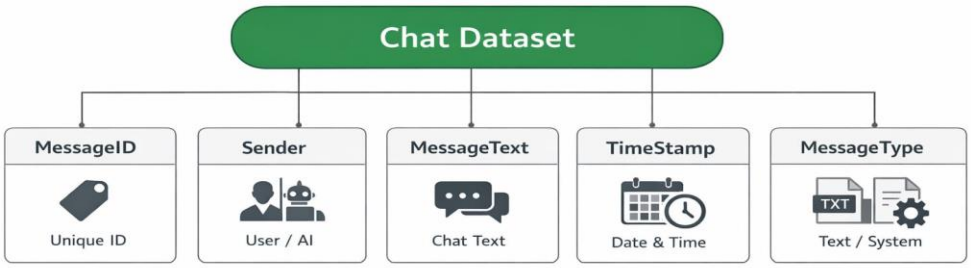
Overview:

In a ChatGPT-style application, there is no fixed traditional dataset stored inside the app. Instead, the system uses pre-trained datasets that were already used to train large language models (like GPT). These datasets include a mixture of:

Text conversations

Articles, books, and websites

Purpose of Dataset:

- To store user messages and AI responses
- To maintain conversation history
- To help the AI give relevant and accurate replies
- To provide a smooth chat experience
- To keep track of past interactions
- To support continuous conversation

Dataset Structure: